Mock Paper 1B — Computer Systems (H446/01) — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Mark scheme

> General guidance: award marks for any valid equivalent point. Where a question asks for "three" of something, award one mark per distinct correct point up to the maximum. On calculation questions award method marks even if the final answer is wrong, provided correct working is shown.

Question 1 (8 marks)

1(a) [2] (AO1) — 1 + 1: - **CIR**: holds the **current instruction** that has just been fetched, so it can be **decoded** (split into opcode and operand) (1). - **ALU**: carries out the **arithmetic and logical operations** (e.g. add, subtract, compare, shift) during the execute stage (1).

1(b) [3] (AO2) — up to 3: - Pipelining means the processor **fetches, decodes and executes different instructions at the same time** (overlapping stages) (1). - While one instruction is being executed the next is being decoded and a third fetched, so an instruction **completes every cycle** once the pipeline is full, raising throughput (1). - The pipeline must be **flushed/refilled after a branch/jump (or interrupt)** because the pre-fetched instructions are no longer the correct next instructions (1).

1(c) [3] (AO2) — up to 3: - 3.2 GHz means the clock generates **3.2×10^9 cycles per second**, i.e. that many fetch–decode–execute steps can be initiated per second (1). - Increasing clock speed raises **heat output and power consumption**, which can cause instability/throttling (1). - Other factors limit performance — e.g. memory/cache speed, number of cores, bus width — so a faster clock alone may leave the CPU **waiting for data** (1). (Accept: not all tasks parallelise / diminishing returns.)

Question 2 (7 marks)

2(a) [3] (AO1) — 2 for distinction, 1 for examples: - **Primary** storage is **directly accessible by the CPU** and is usually volatile and fast (e.g. **RAM**; accept cache/registers) (1). - **Secondary** storage is **non-volatile**, larger, slower and **not directly accessed by the CPU** — used for long-term storage (e.g. **HDD/SSD**) (1). - One valid example of each (1).

2(b) [2] (AO2) — 1 + 1: - **Magnetic tape** (1). - Justification: lowest **cost per gigabyte**, very **high capacity** and long shelf life; slow sequential access is acceptable because the data is rarely retrieved (1).

2(c) [2] (AO2) — 1 + 1: - It is **non-volatile, large-capacity storage held remotely (on the provider's servers/disks)** and not directly addressed by the CPU, so it behaves as secondary storage (1). - One disadvantage: requires an **internet connection** / depends on a third party / potential **privacy/security** or ongoing cost concerns (any one) (1).

Question 3 (9 marks)

3(a) [4] (AO2) — up to 4: - Each process is given a fixed **time slice / quantum** of processor time (1). - Processes are held in a **queue**; when a process's quantum expires it is **pre-empted** and moved to the back of the queue, and the next process runs (1). - This continues cyclically until all processes complete (1). - Advantage over FCFS: **no process can monopolise the CPU**, so short/interactive jobs are not stuck behind a long one — better responsiveness/fairness (1).

3(b) [3] (AO1) — up to 3: - The keyboard sends an **interrupt signal** to the processor requesting attention; checked at the end of the current FDE cycle (1). - The processor **saves the current state/registers/PC onto the stack** so the running program can resume later (1). - The relevant **interrupt service routine** reads the keypress; afterwards the state is **popped off the stack** and the interrupted program continues (1).

3(c) [2] (AO1) — 1 + 1: - A **device driver** is software that allows the OS to **communicate with and control a specific hardware device** (1). - It is needed because each device has different signals/commands; the driver **translates OS instructions into device-specific commands**, so the OS does not need to know the internal details of every device (1).

Question 4 (9 marks)

4(a) [3] (AO1) — up to 3: - An **assembler translates assembly language into machine code** (object code) (1), typically with a **one-to-one** relationship between assembly instructions and machine-code instructions (1). - Assembly is **low-level** because it works directly with the **specific processor's instruction set / registers and memory addresses** (machine-specific, close to hardware) (1).

4(b) [4] (AO2) — 2 + 2: - **Code generation (2):** the parsed/checked program (e.g. abstract syntax tree) is translated into **machine code (object code)** for the target processor. - **Optimisation (2):** the code is improved to **run faster / use less memory** — e.g. removing redundant instructions, simplifying loops, reusing registers — without changing the program's output.

4(c) [2] (AO2) — 1 + 1: - Advantage: the **same intermediate code runs on any platform** with the appropriate virtual machine (portability) / source not exposed (1). - Disadvantage: it generally **runs slower** than native code because the VM must interpret/JIT-compile it at run time, and the VM must be installed (1).

Question 5 (7 marks)

5(a) [4] (AO1) — up to 3 for stages, 1 for sequential point (max 4): - Recognisable stages in order, e.g. **requirements/analysis → design → implementation → testing → maintenance** (up to 3 for naming/describing the stages in sensible order). - It is **sequential** because **each stage must be completed before the next begins**, with formal sign-off, and you do not normally return to a previous stage (1).

5(b) [3] (AO2) — up to 3: - Requirements are **fixed and known in advance** (set by regulation), so Waterfall's up-front requirements capture is appropriate; little need for the rapid iteration RAD provides (1). - Waterfall produces **thorough documentation at each stage**, which is required for regulatory approval/auditing of a medical device (1). - Safety-critical software needs **rigorous, planned testing** against a fixed specification; RAD's emphasis on speed and frequent change could compromise verification/safety (1).

Question 6 (9 marks)

6(a) [2] (AO2) — 1 + 1: - A **class** is a **template/blueprint** that defines the attributes and methods — e.g. `Vehicle` defines `registration`, `maxSpeed` and `describe()` (1). - An **object** is a specific **instance** created from a class with its own attribute values — e.g. a particular minibus with `registration = "AB12 CDE"` (1).

6(b) [4] (AO3) — award: - Private attributes declared (1) - Constructor that initialises both attributes (1) - Public getter for `maxSpeed` (1) - `describe()` returns a string combining both attributes (1)

Example (accept any valid OOP language):

```
class Vehicle
    private registration
    private maxSpeed

    public procedure new(registration, maxSpeed)
        self.registration = registration
        self.maxSpeed = maxSpeed
    endprocedure

    public function getMaxSpeed()
        return self.maxSpeed
    endfunction

    public function describe()
        return self.registration + " (max " + self.maxSpeed + ")"
    endfunction
endclass
```

6(c) [3] (AO2) — up to 3: - `Minibus` is declared as a **subclass/child of** `Vehicle`, so it **inherits** `registration`, `maxSpeed` and `describe()` (1). - It then **adds its own attribute** `seats` (and may add/override methods) (1). - Benefit: **code reuse** — shared code is written once in the base class, reducing duplication and easing maintenance (1).

Question 7 (9 marks)

7(a) [5] (AO3) — trace + output + description:

Inputs: `a = 4`, `b = 3`, `total = 0`, `one = 1`. The loop adds `a` to `total` and decrements `b` each pass, stopping when `b = 0` (BRZ).

Pass	b at test	total += a	b after SUB one
1	3	0 + 4 = 4	2
2	2	4 + 4 = 8	1
3	1	8 + 4 = 12	0
4	0	BRZ → exit	—

- Correct trace of `total` (1) and of `b` decrementing to 0 (1).
- Correct identification that the loop ends via **BRZ when b = 0** (1).
- **Output = 12** (1).
- It performs **multiplication by repeated addition** — i.e. computes `a × b` (4 × 3) (1).

7(b) [4] (AO1) — 1 + 1 + 1 + 1: - **Direct addressing**: the operand holds the **address of the data**; the value at that address is used (1). - **Indirect addressing**: the operand holds the **address of a location that itself contains the address** of the data (a pointer); the processor follows two lookups (1). - Clear distinction drawn between the two (1). - Advantage of indirect: it allows access to a **larger address range than the operand field alone** / supports pointers so the address can be **changed at run time** (e.g. for data structures) (1).

Question 8 (10 marks)

8(a) [3] (AO1) — up to 3: - Frequently occurring symbols/strings are stored once in a **dictionary/table** and replaced in the data by a **shorter code/index** (1). - More common symbols are given **shorter codes** than rare ones (variable-length encoding) (1). - The dictionary is used to **reconstruct the original exactly** on decompression — it is **lossless** (1).

8(b) [4] (AO2) — up to 4: - A hashing algorithm takes an input of any size and produces a **fixed-length value (the hash/digest)** (1). - It is **one-way** — it is computationally infeasible to reverse the hash to recover the original (1). - Passwords are stored as their **hash, not in plain text**; when a user logs in, the entered password is hashed and **compared with the stored hash** (1). - Benefit: if the database is stolen the **actual passwords are not revealed** (accept: salting adds a random value before hashing to defeat precomputed/rainbow-table attacks) (1).

8(c) [3] (AO3) — working + collision + resolution: - 14 MOD 11 = **3**; 25 MOD 11 = **3**; 36 MOD 11 = **3** (1 for correct indices — all map to 3). - **Collision**: 25 (and 36) hash to index 3, already occupied by 14 (1). - Resolution: **open addressing (e.g. linear probing — place at the next free slot)** or **chaining (store a linked list at the index)** — any one valid (1).

Question 9 (10 marks)

9(a) [3] (AO2) — up to 3: - **Referential integrity** ensures that a **foreign key value must match an existing primary key** in the related table (or be null) (1). - It keeps relationships between tables **consistent** (1). - Problem if not enforced: the `Loan` table could contain a `MemberID` for a **member who does not exist** (an "orphaned" loan), so the loan cannot be linked to a valid member (1).

9(b) [3] (AO3) — correct INSERT keyword/table (1), correct columns or value order (1), correct values incl. quoting (1):

```
INSERT INTO Member (MemberID, Surname, JoinDate)
VALUES (207, 'Okafor', '2026-06-27');
```

(Accept `INSERT INTO Member VALUES (207, 'Okafor', '2026-06-27');`.)

9(c) [4] (AO3) — correct fields (1), correct join across both tables (1), correct filter `Returned = 'N'` (1), correct `ORDER BY DueDate` ascending (1):

```
SELECT Member.Surname, Loan.ISBN, Loan.DueDate
FROM Member, Loan
WHERE Member.MemberID = Loan.MemberID
AND Loan.Returned = 'N'
ORDER BY Loan.DueDate;
```

(Accept explicit `INNER JOIN ... ON ...`; accept `ORDER BY DueDate ASC`.)

Question 10 (10 marks)

10(a) [3] (AO1) — up to 3: - A **MAC address** is a **permanent, unique hardware address** assigned to a network interface; used at the **link/network-access layer** for delivery on the local network (1). - An **IP address** is a **logical address** assigned to a device on a network; used at the **internet/network layer** to route packets between networks (1). - Key distinction: MAC is fixed/hardware and local; IP is logical and can change / is used for end-to-end routing (1).

10(b) [4] (AO2) — up to 4: - TCP splits data into **numbered segments/packets** so they can be **reassembled in the correct order** at the destination (1). - The receiver sends an **acknowledgement** for packets received correctly (1). - If an acknowledgement is **not received within a time limit**, the packet is **retransmitted** (1). - **Checksums/error detection** identify corrupted packets so they can be discarded and resent — ensuring reliable, ordered delivery (1).

10(c) [3] (AO2) — up to 3: - **Client–server**: a central **server** provides resources/services that **clients request**; control and data are centralised (1). - **Peer-to-peer**: each device acts as **both client and server**, sharing resources directly with no central server (1). - Client–server suits the website because it gives **central control, security, easier backup/management** and can serve many clients consistently from one managed location (1).

Question 11 (12 marks)

11(a) [3] (AO2) — method + answer + verification: - +53 in binary = `0011 0101` (1). - Invert to `1100 1010`, then add 1 (1). - Result **`1100 1011`** (1). (Verify: −128 + 64 + 8 + 2 + 1 = −53. ✓)

11(b) [5] (AO3) — - Exponent `0010` (two's complement, positive) = **+2** (1). - Mantissa `0.1101000` with the point after the first bit; sign bit `0` so **positive**. Fraction = $2^{-1} + 2^{-2} + 2^{-4}$ = 0.5 + 0.25 + 0.0625 = **0.8125** (1 for mantissa value). - Apply exponent (shift left 2 / multiply by 2^2): 0.8125 × 4 (1 for method). - = **3.25** (1 for final answer). - **Normalised? Yes** — for a positive number the normalised form begins `0` then `1` (`01…`); here the first two bits are `01`, so the leading bits differ and no precision is wasted (1).

11(c) [4] (AO3) —

```
  0101 1010   (=  90)
+ 0011 0110   (=  54)
-----------
  1001 0000   (= 144 unsigned)
```

- Correct bit-by-bit addition with carries (1).
- **8-bit result `1001 0000`** (1).

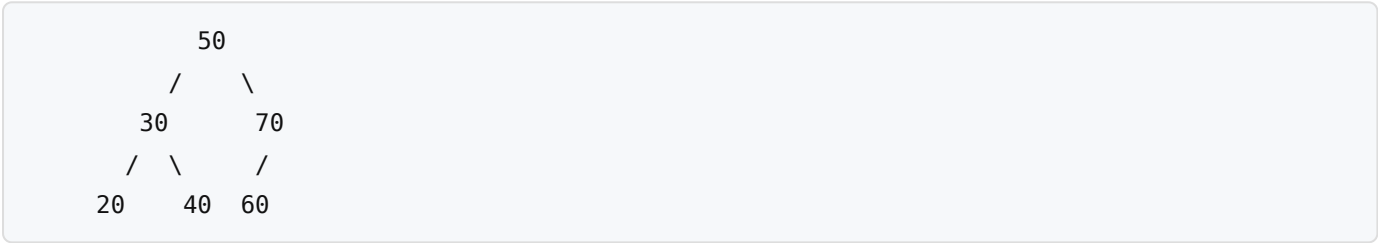
- Interpreted as signed two's complement: `0101 1010` = +90 and `0011 0110` = +54; true sum = +144, which **exceeds the maximum +127** (1).
- Therefore **overflow has occurred** — two positives produced a result with sign bit 1 (`1001 0000` = −112 in two's complement), an incorrect sign (1).

Question 12 (10 marks)

12(a) [3] (AO1) — 1 + 1 + 1: - A **linked list** is a dynamic data structure of **nodes**, each holding **data and a pointer to the next node** (1). - Advantage over a static array: it can **grow/shrink at run time**, and insertion/deletion needs only pointer changes (no shifting) (1). - Disadvantage: it uses **extra memory for pointers** and items **cannot be accessed directly by index** (must traverse from the start) (1).

12(b) [4] (AO3) — tree (2) + in-order traversal (2):

Inserting 50, 30, 70, 20, 40, 60:



- Correct structure: 50 root; 30 left, 70 right; 20 and 40 children of 30; 60 left child of 70 (2 marks; deduct for misplaced node).
- **In-order traversal (left, root, right): 20, 30, 40, 50, 60, 70** (2 marks; this is the keys in ascending order — award 1 if minor slip).

12(c) [3] (AO1) — 1 each: - (i) Insert at head of linked list = **O(1)**. - (ii) Search a balanced BST = **O(log n)**. - (iii) Access array element by index = **O(1)**.

Question 13 (11 marks)

13(a) [4] (AO3) — truth table (3) + gate name (1):

A	B	NOT A	NOT B	Q = NOT A OR NOT B
0	0	1	1	**1**
0	1	1	0	**1**
1	0	0	1	**1**
1	1	0	0	**0**

- 3 marks for a correct Q column (deduct 1 per error; award 1 if at least half correct).
- By De Morgan's law `NOT A OR NOT B = NOT(A AND B)`, so the expression is equivalent to a **NAND gate** (1).

13(b) [4] (AO3) — award 1 per correct step/law, max 4:

```
Q = A.(A + B) + A.C + C
  = A.A + A.B + A.C + C        (distribution)
  = A + A.B + A.C + C          (idempotence: A.A = A)
  = A + A.C + C                (absorption: A + A.B = A)
  = A + C                      (absorption: C + A.C = C)
```

- **Simplest form:** `Q = A + C` (final answer mark).
- (Accept correct alternative ordering; key laws: distribution, idempotence A.A = A, absorption.)

13(c) [3] (AO2) — up to 3: - A flip-flop is a **bistable circuit that stores one bit** (it has two stable states, 0 or 1) (1). - In a sequential circuit it **holds/remembers a value** between clock pulses, so the output depends on past inputs as well as current ones (1). - Many flip-flops together form **registers/memory**, each storing one bit, so a group of *n* flip-flops stores an *n*-bit value — making them the building blocks of registers and static memory (1).

Question 14 (7 marks)

14(a) [3] (AO2) — up to 3: - **Encryption** scrambles the data using a **key** so that, if intercepted, it is **unreadable** without the corresponding key (1). - **HTTPS** adds an encryption layer (TLS/SSL) so data between browser and server is **encrypted in transit** (1). - For online banking this protects **login credentials and financial data** from interception/tampering, unlike plain HTTP which sends data in the clear (1).

14(b) [2] (AO2) — 1 + 1: - **Performance**: it **caches** frequently requested web pages, so repeated requests are served locally/faster and use less bandwidth (1). - **Security**: it acts as an **intermediary that hides clients' IP addresses** and can **filter/block** requests to malicious sites (1).

14(c) [2] (AO1) — 1 + 1: - A **virus** attaches itself to a **host file/program** and requires the user to run/open it to spread (1). - A **worm** is **self-replicating** and spreads **across networks on its own** without needing a host file or user action (1).

Question 15 — Extended response (12 marks) (AO3)

Indicative content (not exhaustive — credit any relevant, well-argued point across legal, ethical, social and stakeholder dimensions):

Legal: - Data Protection Act 2018 / UK GDPR: rights around **automated decision-making** — individuals generally have a right **not to be subject to solely automated decisions** with significant effects, and a right to **human review** and an explanation; automatic deactivation without review may breach this. - Lawful basis for processing location/rating data; data must be accurate and used proportionately. - Employment law: whether drivers are "workers" with rights affects the legality of automated pay/deactivation.

Ethical / moral: - **Lack of transparency** — drivers cannot see how decisions are made ("black box"), undermining fairness and trust. - **Algorithmic bias** — ratings can encode customer prejudice; the algorithm may systematically disadvantage certain groups. - **Accountability** — who is responsible when the algorithm makes a wrong/unfair decision? No human in the loop.

Social: - Power imbalance and job insecurity for gig workers; stress of being managed by an opaque algorithm ("algorithmic management"). - Wider effect of normalising automated hiring/firing across the economy. - Consumer interest in efficient service vs fairness to workers.

Stakeholder perspectives: - **Drivers**: efficient job allocation and pay vs unfair deactivation, no appeal, no transparency. - **Company**: lower management costs, scalability, consistency vs legal/reputational risk and worker resentment. - **Society**: efficient services and innovation vs erosion of workers' rights and accountability.

Conclusion: a justified judgement — e.g. the system may be used **only with safeguards**: transparency about how decisions are made, **human review before deactivation**, an appeals process, and bias auditing; or it should not be used in its current solely-automated form because it likely breaches data-protection rights and is ethically unfair.

Levels of response:

Level 3 (9–12 marks): A thorough discussion covering legal, ethical and social issues with clear application to the scenario. Considers **multiple stakeholders** and weighs competing interests. Reaches a **well-justified, balanced conclusion**. Specialist terminology used accurately; the response is coherent, structured and sustained.

Level 2 (5–8 marks): Discusses several relevant issues with some application to the scenario. Considers more than one viewpoint but with limited balance/depth. A conclusion is offered but **only partially justified**. Some structure and reasonable use of terminology.

Level 1 (1–4 marks): Identifies a few relevant points, largely descriptive/generic with little application to the scenario. One-sided; little or no justified conclusion. Limited structure.

0 marks: No creditable response.

Verified mark total

8 + 7 + 9 + 9 + 7 + 9 + 9 + 10 + 10 + 10 + 12 + 10 + 11 + 7 + 12 = **140 marks**.

Confirmed total: 140 marks.